PNG–Format/Gruppe 17

Datenkompression Pro-Seminar
Dozent: Dr.-Ing. The Anh Vuong
WiSe23/24

Graphische Daten Verarbeitung, Institut für Informatik
Goethe-Universität Frankfurt a. M.

Autor: Oleksandra Popovych, 7377543
Dime Ampov, 6815723
s0231589@stud.uni-frankfurt.de,
s7928961@stud-uni-frankfurt.de

Datum: February 23, 2024

Contents

1 Einführung **2**
1.1 Wichtige Begriffe und Definitionen 3

2 Portable Network Graphics (PNG): Techniken der Bildkompression und -kodierung **4**
2.1 Filterung 4
2.1.1 Filtertyp None: Keine Filterung 5
2.1.2 Filtertyp Sub 5
2.1.3 Filtertyp Up 6
2.1.4 Filtertyp Average 6
2.1.5 Filtertyp Paeth 6
2.1.6 Anwendung der Filterung 7
2.1.7 Ein Beispiel aus der Realität 9
2.2 Der DEFLATE Komprimierungsalgorithmus 9
2.2.1 Die Huffman-Kodierung 10
2.2.2 LEMPEL-ZIV-77 (LZ77) 10
2.2.3 Komprimierungsalgorithmus (DEFLATE) 13

3 Anwendungsgebiet des PNG **14**

4 Qualitätsbewertung des PNG-Verfahrens **15**
4.1 Real-World Vergleiche 15

1 Einführung

Die enormen Mengen an Text, Bildern, Audio- und Videodaten, die täglich über das Internet übertragen werden, führen zwangsläufig dazu, dass die Daten komprimiert werden müssen. Die Datenkomprimierung ist ein Schlüsselkonzept der Informatik, das darauf abzielt, die Größe von Daten zu verringern, um eine effizientere Nutzung von Ressourcen zu ermöglichen. Bevor jedoch eine Komprimierung stattfinden kann, müssen die Daten in verschiedenen digitalen Datenstrukturen gespeichert werden, die eine einfache Visualisierung und Verarbeitung der Informationen ermöglichen. [1]

Die Bilder können als Pixelmatrixdarstellung in verschiedenen Bildformaten wie JPEG, PNG, GIF, BMP, WebP oder anderen gespeichert werden. Bei einigen dieser Formate, wie z. B. dem JPEG-Format, kann es jedoch nach der Komprimierung zu einem gewissen Informationsverlust kommen, was bedeutet, dass die ursprünglichen Daten nicht mehr die gleiche Qualität aufweisen. Im Gegensatz dazu gibt es Formate, bei denen die Ausgangsdaten exakt wiederhergestellt werden können. In diesem Zusammenhang steht das PNG-Format (Portable Network Graphics) im Mittelpunkt dieser Arbeit. [2]

PNG wurde als Antwort auf die Einschränkungen und Lizenzprobleme von GIF entwickelt und bietet einige bedeutende technische Verbesserungen. Solche Vorteile von PNG, wie bessere Farbunterstützung und Transparenz, machen es zu einer besseren Wahl für die Verwendung in Webentwicklung und anderen Anwendungen, wo Qualität und Flexibilität wichtig sind. [3]

1.1 Wichtige Begriffe und Definitionen

- **Bilddaten** - die Darstellung realer Grafiken in digitaler Form, bestehend aus diskreten Einheiten, die als Pixel bezeichnet werden. Dabei werden die Pixel auf ein zweidimensionales Feld von Dateneinträgen eines bestimmten Typs abgebildet. Jeder Eintrag wird in binärer Form dargestellt. [4]

- **Chunks** vermitteln bestimmte Informationen über das Bild. Ein Chunk besteht aus vier Teilen: Länge (4 Byte, Big-Endian), Chunk-Typ/Name (4 Byte), Chunk-Daten (Längenbyte) und CRC (cyclic redundancy code/Prüfsumme; 4 Byte). [3]

- **PNG Signature** - Header der PNG-Datei, 8 Byte lang. [3]

- **RGB-Modell** - Farbdarstellungsmodell, bei dem davon ausgegangen wird, dass alle Farben als eine Kombination aus Rot, Grün und Blau dargestellt werden können. [4]

- **Graustufenbild** - Ein Bild mit 8-Bit-Farbtiefe, das mehrere Intensitätsstufen verwendet, um die Schattierung zwischen Schwarz und Weiß darzustellen. Ein Pixel wird durch 8 Bits dargestellt und enthält einen Farbwert, der 256 verschiedene Grautöne darstellt. [4]

- **Palettenbild** - colormapped- oder index-color-Bilder, werden in vier Pixeltiefen unterstützt: 1, 2, 4 und 8 Bit, was einem Maximum von 2, 4, 16 oder 256 Paletteneinträgen(Farbpaletten) entspricht. [3]

- **Alphakanal** - zusätzlich zu den Rot-, Grün- und Blaukanälen definierter Kanal, der festlegt, welche Pixel während des Renderings überblendet werden sollen; wird in PNG zur Unterstützung der Transperenz verwendet. [2]

2 Portable Network Graphics (PNG): Techniken der Bildkompression und -kodierung

Das PNG-Format ist ein Bildformat, das Daten verlustfrei komprimieren kann, so dass die Qualität des komprimierten Bildes mit der Qualität des Originalbildes identisch ist. [2] Die ursprünglichen Bilddaten können dann Bit für Bit exakt rekonstruiert werden. Folglich, sind PNG-Dateien in der Regel wesentlich größer als gewöhnliche JPEG-Dateien, da keine Informationen verworfen werden. [3]

Die Schlüsselelemente, die PNG verlustfrei machen, sind Filterung, Unterstützung von Alphakanälen (Transparenz) und der DEFLATE Komprimierungsalgorithmus. [3] Diese werden im Folgenden beschrieben.

2.1 Filterung

Vor der Kompression wendet PNG Filtermethoden auf die Bilddaten an, um die Effizienz der Kompression zu erhöhen. Diese Filter ändern die Art und Weise, wie die Daten codiert werden, nicht aber den Inhalt selbst. Dies führt zu einer Clusterbildung um ähnliche Werte, wodurch die Datei leichter zu komprimieren ist. Der DEFLATE-Komprimierungsalgorithmus kann später Muster und wiederholte Sequenzen besser verarbeiten. [3]

PNG unterstützt fünf Arten von Filtern:

- None;
- Sub;
- Up;
- Average;
- Paeth. [3]

Ein Beispiel für die Anwendung der Filterung auf zwei Reihen von Pixelwerten ist in Abbildung 1 dargestellt. Die betroffenen Pixel sind blau hervorgehoben.

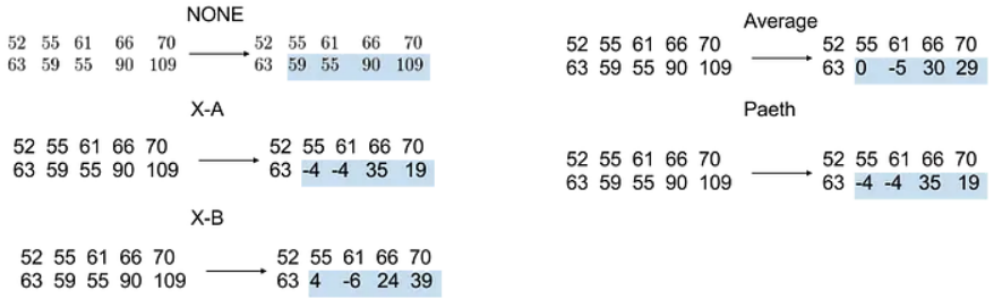

Figure 1: Beispiel für die Anwendung der Filterung
[5]

2.1.1 Filtertyp None: Keine Filterung

Mit dem $None()$-Filter wird die Datenzeile unverändert übertragen. [6]

2.1.2 Filtertyp Sub

Der Filter $Sub()$ überträgt die Differenz zwischen dem aktuellen Byte und dem unmittelbar links daneben liegenden Byte. Um den $Sub()$-Filter zu berechnen, ist die folgende Formel auf jedes Byte der Datenzeile anzuwenden:

$$Sub(x) = Raw(x) - Raw(x - bpp), \tag{1}$$

wobei x im Bereich von null bis zur Anzahl der Bytes, die die Datenzeile minus eins darstellen, liegt. $Raw()$ bezieht sich auf das Rohdatenbyte an dieser Byteposition in der Datenzeile und bpp ist als die Anzahl der Bytes pro vollständigem Pixel definiert, aufgerundet auf eins. [6]

Um die Filterung mit $Sub()$ nach der Dekomprimierung rückgängig zu machen, soll der folgende Wert ausgegeben werden:

$$Sub(x) + Raw(x - bpp) \tag{2}$$

[6]

2.1.3 Filtertyp Up

Der $Up()$-Filter überträgt die Differenz zwischen dem aktuellen Byte und dem darüber liegenden Byte. Um den $Up()$-Filter zu berechnen, wird die folgende Formel auf jedes Byte der Datenzeile angewendet:

$$Up(x) = Raw(x) - Prior(x), \tag{3}$$

wobei $Prior(x)$ sich auf die ungefilterten Bytes der vorherigen Datenzeile bezieht. Für die erste Datenzeile eines Bildes wird $Prior(x) = 0$ für alle x angenommen. [6]
Um die Filterung mit $Up()$ nach der Dekomprimierung rückgängig zu machen, soll der folgende Wert ausgegeben werden:

$$Up(x) + Prior(x) \tag{4}$$

[6]

2.1.4 Filtertyp Average

Der Filter $Average()$ ermittelt die Differenz zwischen dem aktuellen Byte und dem Durchschnitt der entsprechenden Bytes zu seiner Linken und darüber. Um den $Average()$-Filter zu berechnen, wird die folgende Formel auf jedes Byte der Datenzeile angewendet:

$$Average(x) = Raw(x) - floor((Raw(x - bpp) + Prior(x))/2), \tag{5}$$

wobei $Raw()$, $Prior()$, x und bpp wie zuvor definiert sind. [6]
Um die Filterung mit $Average()$ nach der Dekomprimierung rückgängig zu machen, soll der folgende Wert ausgegeben werden:

$$Average(x) + floor((Raw(x - bpp) + Prior(x))/2) \tag{6}$$

[6]

2.1.5 Filtertyp Paeth

Der $Paeth()$-Filter berechnet eine einfache lineare Funktion der drei benachbarten Pixel (links, oben, links oben) und wählt dann als Prediktor

das benachbarte Pixel, das dem berechneten Wert am nächsten liegt. Diese Technik wurde von Alan W. Paeth entwickelt. Um den $Paeth()$-Filter zu berechnen, wird die folgende Formel auf jedes Byte der Datenzeile angewendet:

$$Paeth(x) = Raw(x) - PaethPredictor(Raw(x-bpp), Prior(x), Prior(x-bpp)) \tag{7}$$

Die Funktion $PaethPredictor()$ wird durch den folgenden Pseudocode definiert:

```
function PaethPredictor (a, b, c)
   begin
        ; a = left, b = above, c = upper left
        p := a + b - c        ; initial estimate
        pa := abs(p - a)      ; distances to a, b, c
        pb := abs(p - b)
        pc := abs(p - c)
        ; return nearest of a,b,c,
        ; breaking ties in order a,b,c.
        if pa <= pb AND pa <= pc then return a
        else if pb <= pc then return b
        else return c
   end
```

Um die Filterung mit $Paeth()$ nach der Dekomprimierung rückgängig zu machen, soll der folgende Wert ausgegeben werden:

$$Paeth(x) + PaethPredictor(Raw(x - bpp), Prior(x), Prior(x - bpp)) \tag{8}$$

[6]

2.1.6 Anwendung der Filterung

Ein Encoder kann für jede Pixelreihe des Bildes unterschiedliche Filter verwenden. Es ist wichtig zu beachten, dass die Filterung auf Byte-Ebene und nicht direkt auf Pixeln durchgeführt wird. Dies bedeutet, dass der gewählte Filter auf die einzelnen Farbkomponenten (Bytes) jedes Pixels angewendet wird. In einem Echtfarbenbild, wo jede Farbkomponente (Rot, Grün, Blau) jeweils ein Byte darstellt, wird der Filter nacheinander auf jedes Byte angewendet. Das bedeutet, dass beispielsweise zuerst alle roten Bytes einer Reihe, dann alle grünen Bytes und anschließend alle blauen Bytes miteinander verglichen und verarbeitet werden, wobei alle Farbkomponenten denselben

Filter für die jeweilige Reihe verwenden. Auch wenn die Farbkomponenten verschiedener Pixel unterschiedliche Farbtöne aufweisen, erfolgt der Vergleich bei der Filterung auf Basis ähnlicher Bytes innerhalb derselben Farbkomponente. [3]

Einer der größten Vorteile von PNG ist die Unterstützung von Alphakanälen, die neben den Rot-, Grün- und Blaukanälen auch die Erstellung von Bildern mit variabler Transparenz oder fließenden Übergängen zwischen transparenten und undurchsichtigen Bereichen ermöglichen. [2] Wenn es solche Alphakanäle gibt, werden die Alphabytes beim Filtern immer verglichen. Dabei werden also immer die ähnlichen Werte verglichen, so dass die Kompression so effizient wie möglich erfolgen kann. [3]

Als Beispiel kann ein RGB-Bild herangezogen werden(Abbildung 2). In einem solchen Bild können zwar die Werte für Rot, Grün und Blau unterschiedlich sein, jedoch sind benachbarte Pixel oft sehr ähnlich in ihren Farbwerten. Durch die Anwendung von Filtern werden diese ähnlichen Werte in kleinere Differenzen umgewandelt, was häufig zu Werten nahe Null führt. Nach dem Filterprozess gruppieren sich die transformierten Bytes um den Nullwert. Dies erleichtert die Kompression, da nun ein vorhersehbarer Bereich von Byte-Werten betrachtet wird, was die Effizienz des Kompressionsalgorithmus erhöht. [3]

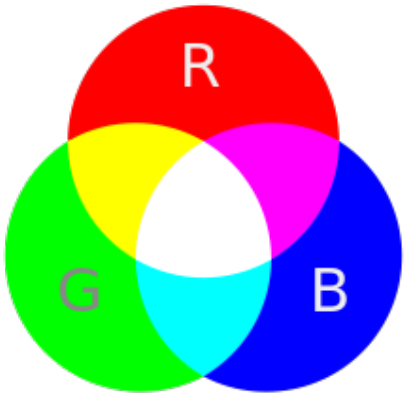

Figure 2: Das RGB-Bild
[7]

Es ist auch eine große Herausforderung für die Entwickler, einen effizienten Algorithmus zu finden, um den richtigen Filter für jede Situation auszuwählen. Das Testen aller möglichen Kombinationen kann sehr zeitaufwändig sein. Die meisten Entwickler verwenden sogenannte Heuristiken, um das Problem zu vereinfachen. Bei einigen Bildformaten ist es kaum sinnvoll, Filter zu verwenden, z. B. bei Palettenbildern oder Graustufenbildern. Bei Graustufen- und Echtfarbbildern mit 8 oder mehr Bits pro Probe, mit oder ohne Alphakanäle, ist die Verwendung unterschiedlicher Filter für verschiedene Teile eines Bildes fast immer von Vorteil. [3]

2.1.7 Ein Beispiel aus der Realität

Die NASA sammelt ozeanographische Daten auf der Ocean ESIP Website. Digitale Karten mit unterschiedlichen physikalischen Abmessungen können dynamisch im GIF- oder PNG-Format erstellt werden, wobei die GIF-Versionen fünfmal so groß sind wie die PNG-Versionen. Dies ist nur dank der PNG-Filterung möglich. So ist beispielsweise eine Karte, die die Oberflächenhöhe des nordöstlichen Pazifiks am 1. August 1997 zeigt, im GIF-Format 70.090 Byte groß, im PNG-Format jedoch nur 13.880 Byte. [6]

2.2 Der DEFLATE Komprimierungsalgorithmus

PNG definiert eine einzige Kompressionsmethode, den Deflate-Algorithmus, für alle Bildtypen. [3] Eine Blockreihe wird als Eingabe für den Algorithmus betrachtet. Jede Blockreihe wird mit einer Kombination aus dem LZ77-Algorithmus und der Huffman-Kodierung komprimiert. Eine zlib-Programmierbibliothek ermöglicht die Komprimierung und Dekomprimierung unter Verwendung des Deflate-Algorithmus. [8]

Obwohl er nicht der beste bekannte Komprimierungsalgorithmus ist, hat Deflate eine sehr attraktive Mischung von Eigenschaften: hohe Zuverlässigkeit, gute Komprimierung, gute Kodiergeschwindigkeit, ausgezeichnete Dekodiergeschwindigkeit, minimaler Überlauf bei nicht komprimierbaren Daten und geringer, gut definierter Speicherbedarf für Kodierung und Dekodierung. [3]

2.2.1 Die Huffman-Kodierung

Der von David Huffman entwickelte Algorithmus ist Bestandteil vieler Datenformate wie ZIP, GZIP und JPEG. Der Algorithmus ordnet jedem Symbol einen Knoten im Binärbaum zu. Die Knoten werden nach der Anzahl der Wiederholungen des Symbols in einer Blockreihe gewichtet. Der Algorithmus kombiniert immer die beiden Knoten mit der niedrigsten Frequenz in einem Bottom-up-Verfahren, um einen Huffman-Baum zu erstellen. Der neue innere Knoten erhält die Summe der Frequenzen der beiden Kindknoten. [8] Die Knoten werden dann mit einer 0 oder 1 gekennzeichnet. Am Ende kann eine Huffman-Kodierung aus dem Huffman-Baum ausgelesen werden. Dadurch erhält man einen schnellen Zugriff auf die am häufigsten vorkommenden Symbole. [4]

```
HUFFMAN(C)
    n = |C|
    Q = C
    for i = 1 to n-1
        allocate a new node z
        x = EXTRACT-MIN(Q)
        y = EXTRACT-MIN(Q)
        z.left = x
        z.right = y
        z.freq = x.freq + y.freq
        INSERT(Q, z)
    return EXTRACT-MIN(Q)
```

[9]

2.2.2 LEMPEL-ZIV-77 (LZ77)

Der LZ77-Algorithmus ist ein grundlegendes Werkzeug der verlustfreien Datenkompression, das auf der Idee basiert, Wiederholungen innerhalb eines Datenstroms zu identifizieren und effizient zu kodieren. Man geht davon aus, dass sich viele Arten interessanter Daten, von gewöhnlichen Text bis hin zu Bildern, in unterschiedlichem Maße wiederholen. Anstatt wiederholte Datenfolgen jedes Mal neu zu schreiben, verwendet LZ77 eine Referenz auf die erste Instanz dieser Sequenz. [8] Vor der Erläuterung des Algorithmus sollten einige weitere wichtige Begriffe eingeführt werden.

Der Algorithmus basiert auf dem Prinzip eines Sliding Windows. Angenommen, es gibt ein Sliding Window, das durch einen Eingabe-String von links nach rechts hin und her bewegt werden kann und ist so konzipiert, dass es die Eingabesequenz dynamisch liest. Das Sliding Window wird in zwei Hälften geteilt, wobei die eine Hälfte ein History-Buffer mit den zuletzt gesehenen Symbolen ist und die andere Hälfte - ein Lookahead-Buffer mit der noch zu verschlüsselnden Sequenz. [4]

Wenn ein Präfix einer String in einer anderen String als Teilstring vorkommt, nennt man dies eine *Präfix-Matching*. Zum Beispiel sind 'B', 'BA' und 'BAB' drei Präfix-Matchings in einer String "ABBBAB". Dabei ist oftmals nur der längste Präfix-Matching von Interesse. Die Anfangsposition des Matching wird durch den Offset bestimmt, der die Position ab dem rechten Rand der String definiert. Diese Definitionen werden im weiteren Verlauf nützlich sein. [4]

Sei H, ein History-Buffer, mit der Länge von l_H Bytes, die bereits verschlüsselt wurden und L, ein Lookahead-Buffer, mit höchstens l_L Symbolen, die noch nicht verschlüsselt wurden. [4] Der LZ77-Algorithmus sieht dann folgendermaßen aus:

- Lese die Eingabezeichen bis L ist voll.
- Durchsuche H von links nach rechts, um eine Präfix-Matching zu finden. Werden mehrere gefunden, wird die längste und am weitesten rechts stehende genommen.
- Wird eine Übereinstimmung der Länge $l \geq 2$ mit dem Offset f gefunden, so wird (f, l, c) ausgegeben, wobei c das erste nicht übereinstimmende Symbol nach der Matching ist.
  Schiebe das Sliding Window(H und L zusammen) f Zeichen nach rechts.
- Falls keine Matching gefunden ist, dann gebe $(0, 0, ASCII(c))$ aus.
  Schiebe das Sliding Window(H und L zusammen) um ein Zeichen nach rechts. [4]

In der Regel besteht ein LZ77-Token aus drei Teilen: Offset, Länge und dem nächsten Symbol im Lookahead-Buffer. Dieses Token wird in der Ausgabe

geschrieben, und das Sliding Window wird um vier Positionen nach rechts verschoben (oder alternativ wird der Eingabestrom nach links verschoben): drei Positionen für die übereinstimmende Zeichenfolge und eine Position für das nächste Symbol. [1]

Als Beispiel wird die Zeichenfolge "abracadabra" in der Tabelle 1 verschlüsselt.

	Offset f	Length l	Symbol c
abracadabra	0	0	'a'
a bracadabra	0	0	'b'
ab racadabra	0	0	'r'
abr acadabra	3	1	'c'
abrac adabra	2	1	'd'
abracad abra	7	4	"

Table 1: LZ77-Beispieltabelle
[8]

Im Vergleich zum Kodierer ist der Dekodierer wesentlich einfacher (LZ77 ist daher eine asymmetrische Komprimierungsmethode). Er muss einen Buffer vorhalten, der so groß ist wie das Window des Encoders. Der Decoder gibt ein Token ein, findet die Übereinstimmung in seinem Buffer, schreibt die Übereinstimmung und das dritte Tokenfeld in den Ausgabestrom und verschiebt die übereinstimmende Zeichenfolge und das dritte Feld in den Buffer. Dies bedeutet, dass LZ77 oder eine seiner Varianten in Fällen nützlich ist, in denen eine Datei einmal (oder nur wenige Male) komprimiert und häufig dekomprimiert wird. [1]

Diese Methode scheint auf den ersten Blick keine Annahmen über die Eingabedaten zu berücksichtigen. Insbesondere achtet sie nicht auf die Häufigkeit der Symbole. Bei näherem Hinsehen zeigt sich jedoch, dass die LZ77-Methode aufgrund der Art des Sliding Windows den Lookahead-Buffer immer mit dem kürzlich eingegebenen Text im Suchbuffer vergleicht und niemals mit Text, der vor langer Zeit eingegeben wurde (und daher aus dem Suchbuffer entfernt wurde). Die Methode geht also implizit davon aus, dass die Muster in den Eingabedaten dicht beieinander liegen. Daten, die dieser

Annahme entsprechen, lassen sich gut komprimieren. [1]

2.2.3 Komprimierungsalgorithmus (DEFLATE)

Durch die Kombination von LZ77 mit einem zusätzlichen Entropie-Kodierung Algorithmus wie der Huffman-Kodierung kann eine deutlich bessere Komprimierungsrate erzielt werden. [8]

Der Deflate-Algorithmus ist eine Variante von LZ77, die auch doppelte Strings in den Eingabedaten identifiziert und die zweite Instanz eines Strings durch einen Verweis auf die vorherige Instanz in Form eines Paares (Distanz, Länge) ersetzt. Die Distanzen sind auf 32K Bytes und die Längen auf 258 Bytes beschränkt. Wenn eine Zeichenfolge nicht in den vorangegangenen 32K Bytes vorkommt, wird sie als eine Sequenz von Literalen ausgegeben. [8]

Es werden zwei Huffman-Bäume erstellt: Literale und Matchlängen werden mit einem Huffman-Baum komprimiert, Distanzen mit einem anderen. Die Huffman-Bäume werden zu Beginn jedes Blocks in einer kompakten Form gespeichert, die beliebig groß sein können, müssen aber im verfügbaren Speicher Platz finden. Ein Block terminiert wenn es sinnvoll ist, einen neuen Block mit frischen Bäumen zu starten. [8]

Für den String-Zugriff wird eine Hash-Tabelle erstellt, die dann alle Eingabe-Strings der Länge 3 enthält. Der Hash-Index wird für die nächsten 3 Bytes berechnet. Wenn die Hash-Kette nicht leer ist, werden alle Strings aus der Kette mit dem Eingabe-String verglichen und der längste Matching wird ausgewählt. Hier wird ein Huffman-Baum von Vorteil, da die Hash-Ketten die aktuellste Strings auswählen, um die kleinsten Distanzen zu priorisieren. [8]

Ein "Lazy Evaluation"-Mechanismus sorgt dafür, dass nach dem Finden einer Matching der Länge N die Länge der Matching auf 1 verkürzt wird und sofort nach dem nächsten Matching gesucht wird. Auf diese Weise muss der Algorithmus nicht auf die nächsten N Schritte warten, um die Suche erneut zu beginnen. [8]

3 Anwendungsgebiet des PNG

Aus der Sicht des Benutzers kann PNG in vielen Anwendungen von Vorteil sein. Das Format ist für die professionelle oder sonstige Bildverarbeitung nützlich, um Zwischenphasen des Bildes zu speichern, ohne dabei an Bildqualität zu verlieren, was beispielweise bei JPEG nicht der Fall ist. Als Rasterformat mit verlustfreier Datenkomprimierung eignet sich PNG gut für detaillierte Grafiken und Diagramme auf Websites, da keine wichtigen Informationen verloren gehen. [3]

Das PNG-Format wurde im Wesentlichen entwickelt, um das GIF-Format im Web zu ersetzen, und verfügt über drei wichtige Vorteile: Alphakanäle, Gamma-Korrekturen und die Möglichkeit, Bilder sehr detailliert darzustellen. Durch die Verwendung von Alphakanälen, die $65,536$ Transparenzstufen unterstützen, können sehr präzise Pixeltiefen erreicht werden, was besonders für die Webentwicklung nützlich ist. Zusätzlich zu den drei Standard-RGB-Bytes für jedes Pixel wird ein viertes Byte eingeführt, das den Unterschied in der Transparenz zwischen denselben Pixeln kennzeichnet. Diese Funktion ist vor allem für kleine Webgrafiken, wie farbige Bullets und ungewöhnlichen Text, sinnvoll. Designer verwenden das PNG-Format oft für Logos, da es transparente Hintergründe unterstützt. [3]

Gamma-Korrekturen in PNG beheben den Differenzwert zwischen der Interpretation der Farbwerte durch Computer (insbesondere Computerbildschirme), sofern die Einstellungen bei verschiedenen Herstellern unterschiedlich sind. Das bedeutet, dass der Nutzer unabhängig vom Gerät immer sehr nahe an der Originalversion des Bildes ist. [3]

Die verlustfreie Bildkompression findet Anwendung in der medizinischen Bildgebung, bei industriellen Anwendungen, der Satellitenbildgebung, bei Echtzeitanwendungen, in Dokumentenverwaltungssystemen, bei der Bildarchivierung, bei HDTV und bei Anwendungen, bei denen die Bildqualität nach der Dekodierung und Rekonstruktion sehr exakt sein muss oder kein Informationsverlust zu erwarten ist. [10]

4 Qualitätsbewertung des PNG-Verfahrens

Das PNG-Format bietet einen portablen, rechtlich unbelasteten, gut spezifizierten Standard zur verlustfreien Komprimierung von Bilddateien. Verlustfrei bedeutet, dass die dekodierte Version der Daten mit der Quellversion dieser Daten identisch ist. Das ist praktisch, weil man eine hohe Qualität beibehalten kann und trotzdem eine gewisse Komprimierung erreicht, die aber nicht annähernd so stark ist wie bei verlustbehafteten Formaten. [2]

Der verlustfreie Natur des PNG-Dateiformats hat nämlich seine Vor- und Nachteile. Auch wenn die Alphakanäle in PNG für die Qualität von Vorteil sind, hat ein vierter Farbkanal seinen Preis, da er die Größe der unkomprimierten Datei erhöht. Was die Qualität betrifft, so erzielt man im Vergleich zum Originalbild pixelgenaue Ergebnisse, aber was die Dateigröße betrifft, so lassen sich komplexe Bilder, die nicht viele gleichfarbige Pixel enthalten, nicht sehr gut komprimieren. [2]

4.1 Real-World Vergleiche

Nur ein Vergleich der zwei Formate anhand echter Bilder kann die Vorteile eines Bildformats gegenüber einem anderen überzeugend aufzeigen. In Tabelle 2 betrachten wir zunächst eine kleine, ausgewählte Gruppe von sieben nicht-animierten GIF-Bildern.

Name	GIF Size (Bytes)
pic 1	38,280
pic 2	1,249
pic 3	298,529
pic 4	20,224
pic 5	4,584
pic 6	1,226
pic 7	27,660
TOTAL	391,752

Table 2: GIF Size
[3]

Nach der Konvertierung in PNG ist bei den meisten Dateien eine Reduzierung der Größe festzustellen. Die folgende Tabelle 3 zeigt die Ergebnisse.

Name	PNG Size (Bytes)	Change
pic 1	35,224	-8.0%
pic 2	722	-42.2%
pic 3	283,839	-4.9%
pic 4	20,476	+1.2%
pic 5	4,812	+5.0%
pic 6	566	-53.8%
pic 7	20,704	-25.1%
TOTAL	**366,343**	**-6.5%**

Table 3: PNG Size and Change Comparison
[3]

Fünf der Bilder wurden bei der Konvertierung in PNG verkleinert - drei davon ganz wesentlich -, während zwei größer wurden. Insgesamt erzielte der Datensatz eine Verbesserung der Bytegröße um 6,5 %. Der Grund dafür, dass zwei Dateien vergrößert wurden, liegt daran, dass detaillierte Bilder mit einer hohen Farbtiefe oder solche mit zusätzlicher Alphakanaltransparenz nach der Konvertierung größer werden können. Auch die Implementierung

von PNG-spezifischen Merkmalen wie Filtermethoden und Interlacing kann die Dateigröße beeinflussen.

References

[1] David Salomon. *A Concise Introduction to Data Compression*, chapter 1 and 3, pages 5–9 and 176–177. UTICS, 2008.

[2] Colt McAnlis and Aleks Haecky. *Understanding Compression*, chapter 12. O'Reilly Media, Inc., 2016.

[3] Greg Roelofs. *PNG: The Definitive Guide*, chapter 9. O'Reilly Media, Inc., 1999.

[4] Ida Mengyi Pu. *Fundamental data compression*, chapter 4 and 7, pages 67–72 and 129–136. Oxford [u.a.] : Butterworth-Heinemann, 2005.

[5] Colt McAnlis. An example of applying filtering to two rows of pixel values. `https://miro.medium.com/v2/resize:fit:720/format:webp/1*x-yXQnUD4Vjjq1OALPyirQ.png`, 2016. Zugriff am: 08.12.2023 16:20 CEST.

[6] Greg Roelofs. libpng. `http://www.libpng.org/pub/png/pngintro.html`, 1996. Zugriff am: 22.02.2024 14:40 CEST.

[7] Immanuelle(Wikipedia user). A diagram demonstrating additive colour with rgb. `https://commons.wikimedia.org/wiki/File:Venn_diagram_rgb.svg`, 2023. Zugriff am: 08.12.2023 14:00 CEST.

[8] Savan Oswal, Anjali Singh, and Kirthi Kumari. Deflate compression algorithm. *International Journal of Engineering Research and General Science*, 4(1):430–436, 2016. Zugriff am: 08.12.2023 15:00 CEST.

[9] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to algorithms*, chapter 15.3, pages 357 – 360. Cambridge, Massachusetts ; London, England : The MIT Press, 2022.

[10] Madhuri A. Joshi, Mehul S. Raval, Yogesh H. Dandawate, Kalyani R. Joshi, and Shilpa P. Metkar. *Image and Video Compression*, chapter 2, page 5. CRC Press, 2014.